

SUNDAY, SEPTEMBER 27

12 Creating agents · 13 Managing security risks

Kerry Back

This morning

Creating agents

The API underneath everything, and an agent with a tool list you chose.

Managing security risks

Believing it, sending data out, letting it act — and what you take back to work.

12 · CREATING AGENTS

A chatbot produces text

It receives text and returns text. That is the entire capability.

- Everything it “knows” arrived as text in the request
- Everything it “does” leaves as text in the response
- No files, no network, no memory between conversations

A chatbot that appears to remember you is being handed the earlier conversation again on every turn.

- 1 You send a request. The harness also sends a list of tools and what each one does.
- 2 The model answers with text, or with a request to use a tool.
- 3 The harness runs the tool and puts the result back into the conversation.
- 4 Repeat until the model answers with text and no tool call.

The loop in step 4 is the whole mechanism. Nothing else is added.

An agent is a chatbot with tools

What the model is actually sent

Every turn, the harness assembles one block of text:

Fixed each turn

The system prompt. The tool definitions.
Any instruction files the harness decided to include.

Growing each turn

Your messages, the model's replies, and
the full output of every tool that has run so far.

A tool result is text in the conversation, which is why a document the agent read can change what it does next.

THE API

A fourth door

DOOR	WHO IS AT THE KEYBOARD	WHAT IT IS FOR
Chat in a browser	you	one question
Claude Code	you	work on a machine
A connector	you, inside another tool	reaching your data
The API	a program	the same operation, many times

The first three all assume a person is present. The fourth is the one that does not.

Why call it directly

Repetition

The same judgment applied to five thousand rows. Nobody is going to paste them one at a time.

Software

The model becomes a component inside something else, with your instructions fixed and the user's input as the variable.

Control

You pick the model, the temperature, and the system prompt, and they do not change under you.

Cost

You pay per token rather than per seat, which is cheaper for bursts and dearer for constant use.

The request

```
1 import anthropic
2
3 client = anthropic.Anthropic()           # reads ANTHROPIC_API_KEY
4
5 resp = client.messages.create(
6     model="claude-sonnet-5",
7     max_tokens=500,
8     system="You classify customer complaints. Answer with one word.",
9     messages=[{"role": "user", "content": "The unit arrived cracked."}],
10 )
11
12 print(resp.content[0].text)
```

Everything in that call is something you chose: the model, the ceiling on output, the instructions, and the input.

The key

- 1 Create a key in the console. It is shown once.
- 2 Put it in an environment variable, not in the code.
- 3 Never commit it. A key in a public repository is found in minutes.
- 4 Set a spending limit on the key before you use it in a loop.

Step 4 is the one people skip. A loop over a large file with a bug in the exit condition spends real money.

What you are charged for

Input tokens and output tokens, priced separately, per million.

Input

everything you send — instructions, tool definitions, the whole conversation so far

Output

only what comes back

A conversation costs more each turn because the entire history is sent again every time. In an agent loop, every tool result joins that history.

First calls

EXERCISE 18 · 30 MINUTES

- 1 Put your key in the environment and make one call that returns a sentence.
- 2 Change the system prompt and make the same call. Note what moved.
- 3 Run the same request three times at temperature 1, then three times at 0.
- 4 Now loop it: classify 200 rows of a CSV, one call per row, and write the results to a new column.
- 5 Report what it cost.

Record the cost of step 4 per row. Fifty thousand rows is that number times two hundred and fifty.

THE LOOP

What an agent is, exactly

A model

Which one, and at what temperature.

A system prompt

What it is for, and what it refuses.

A tool list

Everything it can do.
Nothing else is possible.

A permission mode

Which actions stop for approval.

A place to run

A machine, a directory, a network.

A loop

Call, execute, feed the result back, repeat.

There is no seventh ingredient.

Three kinds of tools

	WHERE THE CODE IS	EXAMPLE
Built in	in the harness	read a file, run a command, fetch a page
Yours	a function you wrote	look up a customer in your system
Someone else's	an MCP server	the Northwind database, EIA, FRED

All three arrive at the model the same way: a name, a description, and a parameter schema.

A tool you write

```
1 @tool("order_total",
2     "Total revenue for one order. Takes an order id.",
3     {"order_id": int})
4 async def order_total(args):
5     df = pd.read_excel("northwind_orderdetails.xlsx")
6     rows = df[df.OrderID == args["order_id"]]
7     total = (rows.UnitPrice * rows.Quantity).sum()
8     return {"content": [{"type": "text", "text": f"${total:,.2f}"]}]}
```

The description is the interface. The model never sees the code.

Build one

EXERCISE 19 · 75 MINUTES · THREE STAGES

- 1 A chatbot with no tools at all. Ask it how many files are in the directory it is running in. Read the answer carefully.
- 2 Give it one built-in tool — reading files — and ask again.
- 3 Write a tool of your own over data you brought or the order spreadsheets. Give it a system prompt that says what it is for.

In stage 1 the model has no way to see the directory. Keep its answer — you compare it against stage 2.

What stage 1 shows

Asked a question about a filesystem it cannot reach, a model with no tools still answers.

- It produces a plausible number
- It produces plausible filenames
- Nothing in the output is marked as unavailable

The same model with a file-reading tool answers correctly. What changed is not the model. It is that something connected the question to the directory.

CHOOSING WHAT IT CAN DO

Narrow beats capable

What you leave out of the tool list is not a limitation. It is the security model.

	A GENERAL AGENT	A NARROW AGENT
Tool list	everything	three tools
Runs as	you	a service account
Reaches	your whole machine	one dataset
Right for	your own work	anything you hand to someone else
Buy for yourself; build for other people.		

An agent whose tool is an agent

- 1 An orchestrator receives the question and holds no data of its own.
- 2 It calls a research agent that can read the public web and nothing private.
- 3 It calls a data agent that can read your database and cannot reach the internet.
- 4 It assembles the answer from what both returned.

The two workers never share a context. Neither one holds private data and an outbound channel at the same time — which is the point of the arrangement, and the subject of the rest of this morning.

Talk it out of its job

EXERCISE 20 · 25 MINUTES

- 1 Take the narrow agent you just built.
- 2 Ask it something outside its purpose. See whether the system prompt holds.
- 3 Now ask persuasively — a plausible reason, an appeal to an exception, an instruction that claims to come from you.
- 4 Note which refusals came from the prompt and which came from a tool that does not exist.

Keep what you found. Some of it is the first item after the break.

- What your agent is for, in one sentence
- Its tool list, read out loud
- The one thing you could talk it into # 13 · Managing security risks

Believing it, sending it out, and letting it act.

Screen shares

Three risks

Believing it

A wrong number, delivered fluently. Friday morning was this one.

Sending data out

Your material reaching a third party. A contract problem.

Letting it act

An agent doing something on your behalf that you did not ask for. An engineering problem.

Most corporate AI policy addresses the second one only.

SENDING DATA OUT

What a contract gives you

	WHAT IT ADDS	ENOUGH FOR
Opt out of training	a setting, revocable, data still retained briefly	personal, non-sensitive work
A signed DPA	liability, breach notice, audit rights, a subprocessor list	client data, regulated data
A DPA plus a BAA	the same, covering protected health information	anything under HIPAA
An enterprise agreement	SLAs, support, SSO, capacity	scale and operations
The last row buys operations. The privacy terms are the same as the row above it.		

What the regulation actually requires

HIPAA

Satisfactory assurances “must be documented through a written contract” meeting § 164.504(e).

A BAA is required, not advisable. 45 CFR 164.502(e).

SEC REG S-P

Written policies “reasonably designed to require oversight, including through due diligence and monitoring, of service providers.”

17 CFR 248.30.

Run it where the data already is

	AWS	GOOGLE CLOUD	AZURE
Claude	Bedrock	Vertex AI	Foundry
GPT	Bedrock		Azure AI Foundry
Gemini		native	

If the data already sits in one of those, the model can run inside the same perimeter. No new vendor, no new trust boundary, and the DPA you already signed still applies.

When the contract is not enough

- 1 Send less — a schema and a few synthetic rows rather than the table.
- 2 Anonymize — strip the names, the account numbers, the identifiers.
- 3 Run a smaller model on your own hardware, so nothing leaves at all.

A DPA is a promise backed by liability. These three are physical facts about where the data went.

LETTING IT ACT

Prompt injection

The model cannot tell an instruction from a piece of content. Both arrive as text in the same conversation.

Text placed where the agent will read it — an email, a document, a web page, a spreadsheet cell — is read in the same way as text you typed. An attacker needs no credentials, only a way to get words in front of the model.

OWASP has ranked this first among risks to LLM applications since 2025.

EchoLeak

CVE-2025-32711 · MICROSOFT 365 COPILOT · JUNE 2025

- 1 An attacker mails the victim a message containing instructions invisible to a human reader.
- 2 The victim asks Copilot to summarize their email. Copilot reads the hidden instructions.
- 3 Following them, Copilot places confidential data inside a URL in its response.
- 4 The victim's browser fetches that URL automatically.
- 5 The attacker reads the data out of their own server log.

Zero clicks by the victim beyond the summary request.

Three legs

Private data

Something worth taking:
client records, financials,
an inbox.

Untrusted content

Text an outsider can
control: a web page, an
email, an uploaded file.

An outbound channel

Any way for text to leave:
fetching a URL, sending
mail, writing somewhere
shared.

All three in one agent is the condition. Remove any one and the attack has nowhere to go.

The question to ask about any deployment

The check

Does this agent hold private data, read content an outsider can influence, and have a way to send text out – at the same time?

It is answerable from an architecture diagram, without knowing anything about how the model works.

Red-team a classmate's agent

EXERCISE 21 · 45 MINUTES

- 1 Swap agents with someone. You get their code and their tool list.
- 2 Diagram it: what private data can it reach, what untrusted text does it read, how could text leave.
- 3 Plant an injection in something it will read — a file, a row, a page it fetches.
- 4 Try to make it act on the injection. Record what worked and what it refused.
- 5 Give the author two changes: one to the prompt, one to the tool list. Say which one you would actually rely on.

Use synthetic data. Nothing in this exercise touches anything real.

MAKING IT IMPOSSIBLE RATHER THAN UNLIKELY

Two ways to stop the same thing

GOVERNANCE

Every call is inspected as it is made and allowed or refused.

Works on the calls you anticipated.
Judgment happens at run time, every time.

ISOLATION

The route does not exist.

Works on calls nobody anticipated.
Nothing is judged, because nothing is possible.

Real systems use both. The order matters: isolate first, then govern what remains.

The split

- 1 The agent that reads untrusted content gets no private data and no credentials.
- 2 The agent that holds private data has no route to the internet.
- 3 An orchestrator calls both as tools and never exposes one to the other's context.

Neither worker ever holds all three legs. That property is a fact about the network and the tool lists, not a rule either agent has been asked to follow.

Your container, as an example

Filesystem	its own — no other student's files exist inside it
Processes	its own — you cannot see anyone else's
Network	its own namespace, and it cannot reach a peer container
The gate	your terminal sits behind your own login

You spent four days inside a working example of the pattern.

MONDAY

Write the plan

EXERCISE 22 · 40 MINUTES

One page, covering:

- 1 The workflow. Who does it now, how often, and how long it takes.
- 2 What you would build — a skill, an app, or an agent — and who uses it.
- 3 The data it touches, and where that data lives today.
- 4 The tool list, written out. What it can do and what it deliberately cannot.
- 5 The three-leg check, answered.
- 6 What it costs to run, and what has to be true for it to be worth that.

35 MINUTES

Three minutes each. The room responds as the people who would have to approve it, fund it, or use it.

For each plan, someone in the room names:

- The step that will not survive contact with the real workflow
- The one item on the tool list they would remove

Pitches

What you have

A habit

Describe the data, require the code, check the counts, reconcile a second way.

A file

A skill, published, that a colleague can install in two commands.

A question

Private data, untrusted content, an outbound channel — asked of anything before it is switched on.